

AstraNotes — Initial Requirement Set

Student Name: Jonathan Fong

Date: April 2, 2026

Project: AstraNotes

1. Architecture Decision Summary

The architecture selected for AstraNotes is a modular C++ design following MVC. The five core components are: a Note model hierarchy with TextNote, VoiceNote, and SecureNote as concrete types; NoteManager as the controller-level coordinator for note lifecycle operations; StorageInterface with a FileStorage implementation so persistence is decoupled from business logic; NoteFactory as the extensibility boundary for adding new note types without modifying existing code; and EncryptionEngine as an isolated service for secure note privacy.

This direction was chosen because it makes ownership, privacy, and failure handling explicit at the design level rather than deferring them to implementation. Every module can be tested independently through injected interfaces, and the architecture can accommodate new note types, storage backends, or UI layers without rewriting existing code.

2. Initial Requirements for the Quarter Project

Functional Requirements

- The system shall allow a user to create a note of type TextNote, VoiceNote, or SecureNote, each identified by a system-generated UUID and a user-supplied title.
- The system shall allow a user to edit the title or body of an existing TextNote, with the last-modified timestamp updated automatically on each change.
- The system shall allow a user to delete a note by its UUID. If the UUID does not exist, the operation shall return a failure status without throwing an exception or crashing.
- The system shall persist each note to local storage as a JSON file and load all previously saved notes when the application starts.
- The system shall reconstruct each note into its correct concrete type (TextNote, VoiceNote, or SecureNote) during load using a factory, without requiring changes to existing note classes when new types are added.
- The system shall allow a user to read the content of a SecureNote only after supplying the correct passphrase. Plaintext shall not be stored in any data member or written to any file.
- The system shall allow a user to search or filter notes by title substring over the in-memory collection.

- The system shall support note metadata including created date, last modified date, note type, and note identifier.

Non-Functional Requirements

- The system shall support a local collection of up to 10,000 notes with constant-time average lookup by UUID, without performing a full linear scan on create, retrieve, or delete operations.
- The system architecture shall separate the note model, storage logic, and controller so each component can be reviewed or tested independently.

Security, Privacy, Reliability, and Governance Requirements

- Secure note content shall be encrypted before being written to disk. The serialized form shall contain only ciphertext. There shall be no code path that writes plaintext to a file or log.
- The system shall handle a corrupt or unreadable note file during startup by quarantining that file and continuing to load all remaining notes, without crashing or silently discarding data.
- The system shall be structured so that every module with an external dependency can be tested using a mock or stub. No unit test shall require write access to the real filesystem or a real encryption library.
- Any external library introduced into the project shall include a written justification covering what the project could not achieve without it, the tradeoffs versus a standard library alternative, and its licensing and supply-chain risk.